

| DS | Session | Sorting | Question Information | Level 1 | Challenge 20 |
|----|---------|---------|----------------------|---------|--------------|
|----|---------|---------|----------------------|---------|--------------|

Problem Description:

**Real estate** is property consisting of land and the buildings on it, along with its **natural resources** such as crops, minerals or water; immovable property of this nature; an interest vested in this (also) an item of real property, (more generally) buildings or **housing** in general.

There are 'n' flats in the village Nelvayal. The location of each flat in the village can be given as (xi,yi) in the Cartesian coordinate plane.

There are "hi" persons living in the i-th flat. Central electricity authority of the village is set to built a wire line across the village.

The wire line is supposed to constructed in a way such that it is the north-east direction.

In other words the wire line is parallel to the line  $y=x$ .

Given that the construction of such line is considered to be effective only if the number of persons living in its left and right side are equal, can you tell if the construction of such wire line is possible?

Constraints:

1 ≤ t ≤ 100  
2 ≤ n ≤ 2\*10^3  
-10^3 ≤ xi, yi ≤ 10^3  
1 ≤ hi ≤ 10^3

It is guaranteed that no two flats are at the same location.

Input Format:

The first line contains a single integer "t" denoting the number of test cases.

The first line of each test case contains 'n' i.e the number flats in the village.

Next 'n' lines contains 3 space-separated integers xi, yi, hi

Output Format:

Print the output in a separate lines contains 't' lines each containing a "YES" or "NO"

Question No:10

Logical Test Cases

| Test Case 1   | Test Case 2   |
|---------------|---------------|
| INPUT (STDIN) | INPUT (STDIN) |
| 3             | 3             |
| 3             | 3             |

```
-2 1 1
-1 1 3
1 -1 4
3
-2 1 1
-1 1 1
-2 1 1
4
-2 1 1
-1 1 2
-2 1 1
3 1 5
```

EXPECTED OUTPUT

YES  
NO  
NO

```
1 -1 4
-1 1 3
1 -1 4
3
1 -1 4
-1 1 1
1 -1 4
4
1 -1 4
-1 1 2
1 -1 4
3 1 5
```

EXPECTED OUTPUT

NO  
NO  
NO

#### ▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t-->0)
```

Test Case 4

KEYWORD

```
int mid = (l+r)/2;
```

Test Case 2

KEYWORD

```
for(int i = 0;i < n;i++)
```

Test Case 3

KEYWORD

```
while(l<= r)
```

#### ▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

14

Test Case 2

TOKEN COUNT

365

Test Case 3

NLOC

60

## Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct {
5      long long pos;
6      long long people;
7  } Hill;
8
9  int compare(const void *a, const void *b) {
10     Hill *h1 = (Hill *)a;
11     Hill *h2 = (Hill *)b;
12
13     if (h1->pos < h2->pos)
14         return -1;
15     if (h1->pos > h2->pos)
16         return 1;
17
18     return 0;
19 }
20
21 int main() {
22     int T;
23     scanf("%d", &T);
24
25     while (T--) {
26         int N;
27         scanf("%d", &N);
```

```
28
29     Hill hills[N];
30     long long total = 0;
31
32     for (int i = 0; i < N; i++) {
33         long long x, y, h;
34
35         scanf("%lld %lld %lld", &x, &y, &h);
36
37         hills[i].pos = x - y;
38         hills[i].people = h;
39
40         total += h;
41     }
42
43     qsort(hills, N, sizeof(Hill), compare);
44
45     long long left = 0;
46     int possible = 0;
47
48     for (int i = 0; i < N; ) {
49         int j = i;
50
51         while (j < N && hills[j].pos == hills[i].pos) {
52             j++;
53         }
54
```

```

55      /*
56      * A line cannot pass through a hill.
57      * Therefore, check the gap before this group.
58      */
59      if (left * 2 == total) {
60          possible = 1;
61          break;
62      }
63
64      for (int k = i; k < j; k++) {
65          left += hills[k].people;
66      }
67
68      i = j;
69  }
70
71      /*
72      * Check the gap after the last hill.
73      */
74      if (left * 2 == total) {
75          possible = 1;
76      }
77
78      if (possible)
79          printf("YES\n");
80      else
81          printf("NO\n");
82      }
83
84      return 0;
85  }

```

**Output:**

**Test Case-1:**

```
3
3
-2 1 1
-1 1 3
1 -1 4
YES
3
-2 1 1
-1 1 1
-2 1 1
NO
4
-2 1 1
-1 2 2
-2 1 1
3 1 5
NO
```

## Test Case-2:

```
3
3
1 -1 4
-1 1 3
1 -1 4
NO
3
1 -1 4
-1 1 1
1 -1 4
NO
4
1 -1 4
-1 1 2
1 -1 4
3 1 5
NO
PS C:\Users\kasha\OneDrive\
```

**Keshav Kumar Rauniyar**  
**CH.SC.U4CSE25236**